

Desarrollo de firmware para establecer comunicaciones inalámbricas punto a punto entre dos sistemas embebidos con RTOS: Brazo robot controlado por IMU

Casillas-Velasco Erik, Gil-Leal Alex

INTRODUCCIÓN

El control remoto de actuadores mediante sensores inerciales representa un área de creciente interés en la robótica y los sistemas embebidos. La capacidad de traducir los movimientos del cuerpo humano en acciones mecánicas en tiempo real requiere una arquitectura de firmware que garantice baja latencia, procesamiento concurrente y comunicación inalámbrica confiable.

Este proyecto integra los conceptos fundamentales del curso de Desarrollo de Firmware para Sistemas Embebidos: tareas concurrentes con FreeRTOS, adquisición de datos mediante I2C y ADC, procesamiento de señales con filtros complementarios, generación de PWM para control de servomotores, y comunicación inalámbrica punto a punto mediante el protocolo ESP-NOW. El resultado es un brazo robótico que replica los movimientos del brazo del usuario en tiempo real, utilizando dos ESP32 como unidades maestro (sensores) y esclavo (actuadores).

El sistema fue desarrollado siguiendo una metodología por fases: primero se validó toda la funcionalidad en un solo ESP32, luego se dividió el firmware en dos nodos comunicados inalámbricamente, y finalmente se realizó un proceso sistemático de calibración con mediciones reales del sensor montado en el brazo del usuario.

OBJETIVOS

Objetivo General:

Desarrollar un sistema de firmware distribuido basado en FreeRTOS que permita el control inalámbrico en tiempo real de un brazo robótico de 3 grados de libertad mediante sensores inerciales y un potenciómetro, utilizando comunicación ESP-NOW entre dos ESP32.

Objetivos Específicos:

- Implementar la lectura de un sensor IMU (MPU6050) mediante I2C y un potenciómetro

lineal mediante ADC en una arquitectura multitarea con FreeRTOS.

- Diseñar una arquitectura de tareas con prioridades diferenciadas para adquisición, procesamiento, transmisión y monitoreo de datos.
- Establecer comunicación inalámbrica punto a punto entre dos ESP32 mediante el protocolo ESP-NOW con latencia inferior a 5 ms.
- Desarrollar un proceso de calibración sistemático basado en mediciones reales para mapear las lecturas del sensor a posiciones de los servomotores.
- Documentar las limitaciones físicas del sistema derivadas del montaje del sensor y proponer soluciones implementadas.

MARCO TEÓRICO

Arquitectura Maestro-Esclavo con ESP-NOW

ESP-NOW es un protocolo de comunicación inalámbrica propietario de Espressif que permite la transmisión directa de datos entre dispositivos ESP32 sin necesidad de un punto de acceso WiFi. Opera sobre la capa de enlace IEEE 802.11, lo que le confiere una latencia típica de 1-2 ms por paquete y un alcance de aproximadamente 30 metros en interiores. En este proyecto se utiliza para transmitir una estructura de 12 bytes (tres valores uint32_t de ~~duty cycle~~) cada 20 ms desde el nodo maestro hacia el esclavo, equivalente a una tasa de actualización de 50 Hz.

Sensor Inercial MPU6050 y Cálculo de Orientación

El MPU6050 integra un acelerómetro triaxial y un giroscopio triaxial con ADCs de 16 bits. El acelerómetro, configurado en rango de +/-2g (sensibilidad de 16384 LSB/g), permite calcular los ángulos de inclinación Pitch y Roll mediante funciones trigonométricas inversas:

$$\text{Pitch} = \text{atan2}(Ax, \sqrt{Ay^2 + Az^2}) \times 180 / \pi$$
$$\text{Roll} = \text{atan2}(Ay, \sqrt{Ax^2 + Az^2}) \times 180 / \pi$$

Estos ángulos se suavizan mediante un filtro complementario de primer orden con factor $\alpha = 0.90$, que pondera 90% el valor anterior filtrado y 10% la nueva lectura, reduciendo el ruido de alta frecuencia del acelerómetro sin introducir un retardo excesivo en la respuesta.

Planificación de Tareas en FreeRTOS

El sistema emplea el planificador preemptivo de FreeRTOS con cuatro tareas en el nodo maestro y dos en el esclavo. La asignación de prioridades sigue el principio de criticidad temporal: la adquisición de datos del sensor (Prioridad 4) tiene la mayor urgencia para garantizar muestreo constante, seguida del procesamiento (Prioridad 3), la transmisión ESP-NOW (Prioridad 2) y finalmente el monitoreo serial (Prioridad 1). Esta jerarquía asegura que el retardo en la impresión serial nunca degrade la frecuencia de muestreo ni la tasa de transmisión.

DESARROLLO

Lista de Materiales

- 2x ESP32
- 1x MPU6050 (modulo IMU)
- 1x Potenciómetro 10K lineal
- 2x Servo MG996R (hombro del brazo robot)
- 1x Servo MG90S (codo del brazo robot)
- 1x Fuente de alimentación 5V / 3A
- Estructura impresa en 3D para el brazo
- Cables, protoboard, cintas de sujeción

Diagramas de Conexiones

Se diseñaron dos circuitos independientes. El ESP32 maestro se conecta al IMU mediante I2C (GPIO 21 para SDA, GPIO 22 para SCL) y al potenciómetro en GPIO 34 (ADC1 canal 6). El ESP32 esclavo genera las señales PWM a 50 Hz con resolución de 16 bits en GPIO 25, 26 y 27 para los tres servomotores, alimentados mediante una fuente externa de 5V/3A con tierra común al ESP32.

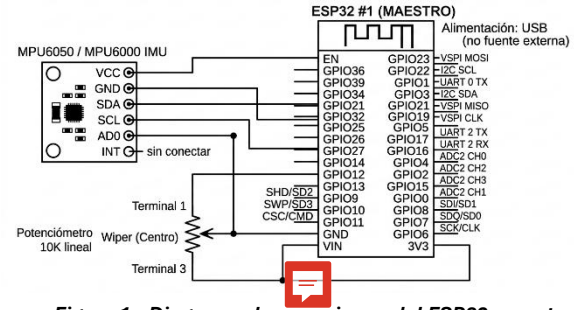


Figura 1.- Diagrama de conexiones del ESP32 maestro (sensores)

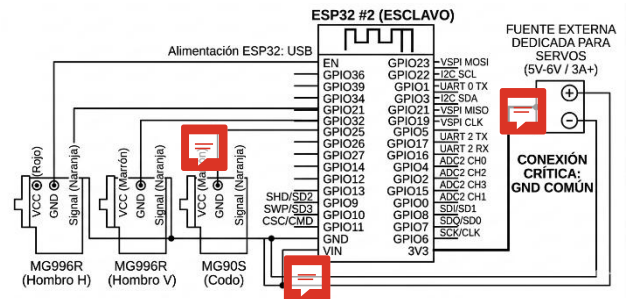


Figura 2.- Diagrama de conexiones del ESP32 esclavo (servos)

Fase 1: Validación en un Solo ESP32

Inicialmente se implementó todo el sistema en un único ESP32 para validar el funcionamiento de los sensores y los servos antes de introducir la complejidad de la comunicación inalámbrica. Este enfoque permitió depurar el cálculo de ángulos, verificar el mapeo de duty cycles y confirmar el rango mecánico de los servomotores sin las variables adicionales del enlace inalámbrico.

Fase 2: División en Dos Nodos con ESP-NOW

Una vez validado el sistema monolítico, se dividió el firmware en dos programas independientes. El maestro conservó la adquisición y el procesamiento, reemplazando la escritura directa de PWM por el envío de una estructura `paquete_servos_t` de 12 bytes vía ESP-NOW. El esclavo recibe esta estructura en un callback y la aplica a los servos.

El protocolo de comunicación se diseñó con un mecanismo de seguridad: si el esclavo no recibe paquetes durante 2 segundos (configurable mediante `TIMEOUT_SIN_DATOS_MS`), mueve automáticamente los tres servos a posición central, previniendo movimientos erráticos ante pérdida de señal.

Durante esta fase se enfrentaron dos problemas técnicos relevantes. Primero, ESP-IDF

v5.5.2 introdujo un cambio en la firma del callback de envío de ESP-NOW: el primer parámetro cambio de `const uint8_t *mac_addr` a `const wifi_tx_info_t *tx_info`, lo cual generaba un error de compilación por tipos incompatibles. Segundo, un cable USB de solo carga impedía la detección del puerto serial del ESP32, problema que se resolvió al verificar con un cable de datos.

Fase 3: Calibración con Mediciones Reales

La calibración represento la fase más crítica del proyecto. Se desarrollo un firmware de diagnóstico independiente cuyo único propósito era leer los valores crudos del acelerómetro (AX, AY, AZ), el giroscopio (GX, GY, GZ), los ángulos calculados (Pitch, Roll) y el potenciómetro, imprimiéndolos cada 500 ms por el monitor serial sin mover servos ni transmitir datos.

Se definieron 8 posiciones de referencia y se registraron las lecturas del sensor en cada una. Adicionalmente, la posición de reposo se midió orientando el cuerpo hacia los cuatro puntos cardinales (Norte, Oeste, Sur, Este) para verificar que el sensor fuera inmune a la orientación magnética. Los resultados confirmaron que el Roll se mantuvo estable en aproximadamente -85 grados independientemente de la orientación cardinal, validando su idoneidad para controlar el eje vertical.



Figura 3.- Dispositivo maestro anclado al brazo derecho en posición de reposo. Se observan el ESP32 con el IMU soldado perpendicularmente y el potenciómetro en la articulación del codo.

Resultados de Calibración

El análisis de las mediciones revelo los siguientes rangos útiles:

Posición	Pitch	Roll	Mapeo
Reposo (abajo)	-4	-85	Roll mínimo
Adelante (horizontal)	-12	+6	Roll medio
Arriba (pidiendo palabra)	-9	+57	Roll máximo
Derecha (Cristo)	-12	-10	Pitch máximo
Cruzado (pecho)	-22	-10	Pitch mínimo
Pot extendido	-	-	ADC = 830
Pot flexionado	-	-	ADC = 2022

Tabla 1.- Valores promedio de Pitch y Roll medidos en cada posición de referencia

El Roll mostro un rango útil de 142 grados (-85 a +57), proporcionando excelente resolución para el control del servo vertical. El Pitch, en cambio, solo vario 17 grados (-25 a -8) entre las posiciones de Cristo y cruzado, resultado directo del montaje perpendicular del IMU respecto al brazo.

ANÁLISIS DE LIMITACIONES Y DECISIONES DE DISEÑO

Limitación del Acelerómetro para Detección de Yaw

Durante las pruebas se identificó una limitación física fundamental: el acelerómetro no puede distinguir entre el brazo extendido hacia adelante y el brazo extendido hacia la derecha (posición de Cristo). En ambos casos, el brazo paso de estar colgando verticalmente a estar horizontal, y los valores de Pitch y Roll resultaron prácticamente idénticos (Pitch = -12, Roll = +6 para adelante versus Pitch = -12, Roll = -10 para Cristo). Esto ocurre porque la rotación alrededor del eje de gravedad (yaw) es invisible para un acelerómetro; la gravedad no cambia al girar horizontalmente.

Este problema es especialmente relevante en una configuración pan-tilt de dos motores: para llevar el brazo robot a la posición de Cristo desde reposo, el servo horizontal debe girar 90 grados primero, pero el acelerómetro no detecta ese movimiento. El servo

vertical si responde correctamente porque el brazo efectivamente cambio de altura.

Solución Implementada: Control Horizontal Limitado

Ante esta limitación, se optó por mantener el acelerómetro como única fuente de datos para ambos ejes, asignando el Roll al servo vertical (rango amplio) y el Pitch al servo horizontal (rango limitado). El movimiento horizontal funciona para los gestos de cruzar el brazo frente al pecho versus extenderlo hacia la derecha, aunque con menor resolución angular. Los rangos de Pitch se calibraron específicamente: PITCH_MIN = -25.0 (cruzado) y PITCH_MAX = -8.0 (Cristo), con sus propios límites de duty independientes.

Se considero implementar el giroscopio para el eje horizontal mediante integración de velocidad angular. Si bien esta solución detectaría la rotación de yaw correctamente, introduce el problema de drift (desviación acumulativa), que requiere mecanismos adicionales como zona muerta y decaimiento. Se implemento un prototipo con giroscopio, pero la solución basada únicamente en acelerómetro resulto más estable para la demostración final.



Figura 4.- Dispositivo maestro durante la flexión del codo. El potenciómetro lineal de 10K acompaña la rotación de la articulación.

ESTRUCTURA DEL FIRMWARE FINAL

ESP32 Maestro: Arquitectura de Tareas

El firmware del maestro implementa cuatro tareas FreeRTOS con prioridades descendentes:

Tarea	Prioridad	Periodo	Función
Adquisición	4 (máxima)	20 ms	Lee IMU (I2C) y pot (ADC)
Procesamiento	3	20 ms	Calcula ángulos, filtra, genera ruties
Envío ESP-NOW	2	20 ms	Transmite 12 bytes al esclavo
Monitor	1 (mínima)	500 ms	Imprime datos por serial

Tabla 2.- Tareas FreeRTOS del ESP32 maestro

Todas las variables de calibración se definen como constantes preprocesador (#define) al inicio del archivo, con comentarios que explican el efecto de modificar cada valor. Cada servo posee sus propios límites de duty independientes (DUTY_MIN_HOMBRO_V, DUTY_MAX_HOMBRO_V, DUTY_MIN_HOMBRO_H, DUTY_MAX_HOMBRO_H, DUTY_MIN_CODO, DUTY_MAX_CODO), permitiendo calibración individual sin afectar a los demás.

ESP32 Esclavo: Recepción y Actuación

El esclavo implementa dos tareas: Salida (Prioridad 3) que aplica los duties recibidos a los servos mediante el periférico LEDC, y Monitor (Prioridad 1) que reporta la tasa de paquetes recibidos por segundo. El esclavo imprime su dirección MAC al arrancar en un formato listo para copiar al código del maestro, simplificando el emparejamiento entre dispositivos.

Protocolo de Comunicación

La estructura del paquete transmitido es de tipo paquete_servos_t, idéntica en ambos nodos, conteniendo tres campos uint32_t: duty_hombro_h, duty_hombro_v y duty_codo. El maestro calcula los duties a partir de los ángulos del sensor y los transmite ya listos para aplicar, minimizando la carga computacional en el esclavo. El callback de recepción del esclavo (espnw_recepcion_cb) verifica el tamaño del paquete antes de procesarlo y actualiza una marca de tiempo para el mecanismo de timeout.



Figura 5.- Brazo robótico impreso en 3D. Se observan los dos servos MG996R formando la articulación del hombro (pan-tilt) y el servo MG90S en la unión entre el vice y el antebrazo.

RESULTADOS

El sistema final logro replicar los movimientos del brazo del usuario en el brazo robótico con una respuesta fluida y perceptiblemente instantánea. La tasa de transmisión ESP-NOW alcanzo los 50 paquetes por segundo, con un porcentaje de entrega exitosa superior al 98% a distancias menores de 5 metros.

Comportamiento por Eje

Eje vertical (Roll, servo del hombro vertical): Funcionamiento excelente. El brazo robot sube y baja siguiendo el movimiento del brazo del usuario con alta fidelidad. El rango de 142 grados de Roll proporciona resolución suficiente para distinguir claramente entre brazo colgando, horizontal y levantado.

Eje horizontal (Pitch, servo del hombro horizontal): Funcionamiento limitado pero operativo. El servo responde al gesto de cruzar el brazo frente al pecho versus extenderlo a la derecha. El rango de 17 grados, aunque estrecho, se mapea al recorrido completo del servo, amplificando el efecto del movimiento.

Codo (Potenciómetro, servo del codo): Funcionamiento correcto. La flexión del antebrazo se reproduce fielmente en el brazo robot. El rango calibrado de 830-2022 del ADC se mapea linealmente a 0-135 grados.

Problemas Encontrados y Soluciones

Problema	Causa	Solución
Error de compilación en ESP-NOW	Cambio de API en ESP-IDF v5.5.2	Actualizar firma del callback a wifi_tx_info_t
ESP32 no detectado en puerto serial	Cable USB de solo carga	Reemplazar por cable USB de datos
Servo horizontal sube en posición de Cristo	El acelerómetro no detecta yaw	Aceptar limitación; mapear Pitch disponible
Potenciómetro no usa rango completo	Montaje físico limita rotación	Calibrar rango efectivo POT: 830-2022

Tabla 3.- Problemas encontrados durante el desarrollo y sus soluciones

LIMITACIONES DEL SISTEMA

Hardware: El IMU soldado perpendicularmente a la placa en posición fija limita la detección de movimientos horizontales (yaw) porque el acelerómetro es físicamente incapaz de medir rotaciones alrededor del eje de gravedad. Un magnetómetro complementario (como el HMC5883L) resolvería este problema, pero no estaba disponible en el hardware.

Mecánica: El brazo robot posee solo 2 grados de libertad en el hombro (pan-tilt), mientras que el hombro humano tiene 3. Esto implica que existen posiciones del brazo humano que el robot no puede replicar, particularmente las que combinan elevación con rotación lateral desde la posición de reposo.

Software: El uso de variables globales compartidas entre tareas sin mecanismos de sincronización es aceptable en este caso porque cada variable tiene un único escritor, pero no es escalable a sistemas más complejos. Adicionalmente, el filtro complementario de primer orden introduce un retardo proporcional al factor alfa; valores muy altos (>0.95) suavizan el ruido, pero ralentizan la respuesta del sistema.

Comunicación: ESP-NOW no implementa retransmisión automática de paquetes perdidos. En ambientes con alta interferencia electromagnética, la tasa de perdida podría incrementarse. El timeout de 2

segundos mitiga este riesgo llevando los servos a posición segura.

CONCLUSIONES

Se desarrollo exitosamente un sistema distribuido de control de un brazo robótico que integra los conceptos centrales del curso: tareas concurrentes con FreeRTOS, comunicación por bus I2C, lectura analógica por ADC, generación de señales PWM, y comunicación inalámbrica punto a punto con ESP-NOW. El sistema demuestra la viabilidad de utilizar dos microcontroladores ESP32 coordinados mediante un protocolo de baja latencia para aplicaciones de control remoto en tiempo real.

La metodología de desarrollo por fases (validación monolítica, división en nodos, calibración sistemática) resulto clave para el éxito del proyecto. La fase de calibración con mediciones reales fue particularmente valiosa: permitió identificar el acoplamiento entre ejes del sensor, determinar los rangos efectivos del potenciómetro y ajustar los parámetros del firmware a las condiciones reales del hardware.

Las limitaciones identificadas, especialmente la incapacidad del acelerómetro para detectar rotaciones de yaw, representan restricciones físicas inherentes al sensor y no deficiencias del firmware. El código se diseñó con variables de calibración expuestas e independientes por cada servo, facilitando ajustes futuros sin modificar la lógica del programa. Esto valida el enfoque de firmware parametrizado que permite adaptar el comportamiento del sistema mediante cambios de configuración en lugar de cambios de código.

REFERENCIAS

- Espressif Systems. (2024). ESP-NOW. En ESP-IDF Programming Guide (v5.5). https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32/api-reference/network/esp_now.html
- Espressif Systems. (2024). LED Control (LEDC). En ESP-IDF Programming Guide (v5.5). <https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32/api-reference/peripherals/ledc.html>
- Espressif Systems. (2024). I2C Driver. En ESP-IDF Programming Guide (v5.5). <https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32/api-reference/peripherals/i2c.html>

- InvenSense. (2012). MPU-6000 and MPU-6050 Product Specification, Revision 3.3. Document Number: PS-MPU-6000A-00.
- Barry, R. (2024). Mastering the FreeRTOS Real Time Kernel. Real Time Engineers Ltd. https://www.freertos.org/Documentation/RTOS_book.html

ANEXO

Código Maestro

/*

BRAZO ROBOT - ESP32 MAESTRO (SENSORES)

Descripción:

Este ESP32 lee el IMU (MPU6050/MPU6000) y el potenciómetro 10K, procesa los datos y envía los valores de duty al esclavo via ESP-NOW.

Nota sobre el movimiento horizontal:

Por el montaje perpendicular del IMU, el rango de PITCH para el movimiento horizontal es limitado (~10°). El servo horizontal funcionará pero con menor precisión que el vertical (~140°).

*/

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/i2c.h"
#include "esp_adc/adc_oneshot.h"
#include "esp_wifi.h"
#include "esp_now.h"
#include "esp_netif.h"
#include "esp_event.h"
#include "nvs_flash.h"
```

```
// DIRECCIÓN MAC DEL ESP32 ESCLAVO
static uint8_t MAC_ESCLAVO[] = {0xFF, 0xFF,
0xFF, 0xFF, 0xFF, 0xFF}; // ← CAMBIAR
```

```
// VARIABLES DE CALIBRACIÓN //
// Configuración del IMU
#define MPU_ADDR          0x68
#define I2C_MASTER_NUM    I2C_NUM_0
#define I2C_SDA_PIN        21
#define I2C_SCL_PIN        22
```

```
#define I2C_FREQ_HZ          400000

// Configuración del ADC (Potenciómetro)
#define POT_ADC_UNIT        ADC_UNIT_1
#define POT_ADC_CHANNEL     ADC_CHANNEL_6 //
GPIO 34

// SERVO VERTICAL DEL HOMBRO (MG996R #1 -
movimiento arriba/abajo)
// Controlado por ROLL del acelerómetro.
// Mediciones: brazo abajo ROLL~-85, horizontal
ROLL~+6, arriba ROLL~+57
#define ROLL_MIN            -85.0 // ROLL
cuando el brazo está colgando (abajo)
#define ROLL_MAX            57.0 // ROLL
cuando el brazo está arriba (pidiendo palabra)
#define DUTY_MIN_HOMBRO_V   1638 // Duty
mínimo del servo vertical (pulso 0.5ms)
#define DUTY_MAX_HOMBRO_V   8192 // Duty
máximo del servo vertical (pulso 2.5ms)
#define INVERTIR_VERTICAL   0

// SERVO HORIZONTAL DEL HOMBRO (MG996R #2 -
movimiento derecha/izquierda)
// Controlado por PITCH del acelerómetro.
// Mediciones: Cristo(derecha) PITCH~-10,
cruzado(izquierda) PITCH~-25
// NOTA: Rango limitado (~15°) por el montaje
perpendicular del IMU.
#define PITCH_MIN           -25.0 // PITCH
cuando el brazo cruza el pecho (izquierda)
#define PITCH_MAX           -8.0 // PITCH
cuando el brazo está extendido (derecha/Cristo)
#define DUTY_MIN_HOMBRO_H   1638 // Duty
mínimo del servo horizontal (pulso 0.5ms)
#define DUTY_MAX_HOMBRO_H   8192 // Duty
máximo del servo horizontal (pulso 2.5ms)
#define INVERTIR_HORIZONTAL 0

// SERVO DEL CODO (MG90S - flexión del
antebrazo)
// Controlado por potenciómetro.
// Mediciones: brazo extendido POT~830, brazo
flexionado 135° POT~2022
#define POT_EXTENDIDO       830 // Valor del
ADC con el brazo extendido (0°)
#define POT_FLEXIONADO      2022 // Valor del
ADC con el brazo flexionado (135°)
#define CODO_ANGULO_MIN     0.0 // Ángulo
mínimo del codo (extendido)
#define CODO_ANGULO_MAX     135.0 // Ángulo
máximo del codo (flexionado)

#define DUTY_MIN_CODO       1638 // Duty
mínimo del servo del codo
#define DUTY_MAX_CODO       8192 // Duty
máximo del servo del codo
#define INVERTIR_CODO       0

// Filtro complementario
#define ALFA_FILTRO          0.90

// Periodos de las tareas (milisegundos)
#define PERIODO_ADQUISICION 20
#define PERIODO_PROCESAMIENTO 20
#define PERIODO_ENVIO        20
#define PERIODO_MONITOR      500

// ESTRUCTURA DEL PAQUETE

typedef struct {
    uint32_t duty_hombro_h; // Duty del servo
horizontal del hombro
    uint32_t duty_hombro_v; // Duty del servo
vertical del hombro
    uint32_t duty_codo; // Duty del servo
del codo
} paquete_servos_t;

// VARIABLES GLOBALES

int16_t g_accel_x = 0; // Aceleración
cruda eje X
int16_t g_accel_y = 0; // Aceleración
cruda eje Y
int16_t g_accel_z = 0; // Aceleración
cruda eje Z
int g_pot_raw = 0; // Valor crudo
del potenciómetro (0-4095)

float g_pitch = 0.0; // Pitch sin
filtrar (horizontal)
float g_roll = 0.0; // Roll sin
filtrar (vertical)
float g_pitch_filtrado = 0.0; // Pitch
suavizado
float g_roll_filtrado = 0.0; // Roll
suavizado

paquete_servos_t g_paquete = {0, 0, 0};
volatile bool g_envio_exitoso = false;
adc_oneShot_unit_handle_t adc_handle;

// FUNCIONES DE INICIALIZACIÓN
```

```
static void init_i2c(void)
{
    i2c_config_t conf = {
        .mode = I2C_MODE_MASTER,
        .sda_io_num = I2C_SDA_PIN,
        .scl_io_num = I2C_SCL_PIN,
        .sda_pullup_en = GPIO_PULLUP_ENABLE,
        .scl_pullup_en = GPIO_PULLUP_ENABLE,
        .master.clk_speed = I2C_FREQ_HZ,
    };
    i2c_param_config(I2C_MASTER_NUM, &conf);
    i2c_driver_install(I2C_MASTER_NUM,
conf.mode, 0, 0, 0);
    printf("[INIT] I2C en SDA=%d, SCL=%d\n",
I2C_SDA_PIN, I2C_SCL_PIN);
}

static void mpu_write_byte(uint8_t reg, uint8_t
data)
{
    i2c_cmd_handle_t cmd =
i2c_cmd_link_create();
    i2c_master_start(cmd);
    i2c_master_write_byte(cmd, (MPU_ADDR << 1) |
0, true);
    i2c_master_write_byte(cmd, reg, true);
    i2c_master_write_byte(cmd, data, true);
    i2c_master_stop(cmd);
    i2c_master_cmd_begin(I2C_MASTER_NUM, cmd,
pdMS_TO_TICKS(100));
    i2c_cmd_link_delete(cmd);
}

static void mpu_read_bytes(uint8_t reg, uint8_t
*buf, size_t len)
{
    i2c_cmd_handle_t cmd =
i2c_cmd_link_create();
    i2c_master_start(cmd);
    i2c_master_write_byte(cmd, (MPU_ADDR << 1) |
0, true);
    i2c_master_write_byte(cmd, reg, true);
    i2c_master_start(cmd);
    i2c_master_write_byte(cmd, (MPU_ADDR << 1) |
1, true);
    if (len > 1) {
        i2c_master_read(cmd, buf, len - 1,
I2C_MASTER_ACK);
    }
    i2c_master_read_byte(cmd, buf + len - 1,
I2C_MASTER_NACK);
    i2c_master_stop(cmd);
```

```
    i2c_master_cmd_begin(I2C_MASTER_NUM, cmd,
pdMS_TO_TICKS(100));
    i2c_cmd_link_delete(cmd);
}

static void init_mpu(void)
{
    mpu_write_byte(0x6B, 0x00); //
Despertar el MPU
    vTaskDelay(pdMS_TO_TICKS(100));
    mpu_write_byte(0x1C, 0x00); //
Acelerómetro
    printf("[INIT] MPU6050/MPU6000
inicializado\n");
}

static void init_adc(void)
{
    adc_oneshot_unit_init_cfg_t init_config = {
        .unit_id = POT_ADC_UNIT };
    adc_oneshot_new_unit(&init_config,
&adc_handle);

    adc_oneshot_chan_cfg_t chan_config = {
        .bitwidth = ADC_BITWIDTH_DEFAULT,
        .atten = ADC_ATTEN_DB_12,
    };
    adc_oneshot_config_channel(adc_handle,
POT_ADC_CHANNEL, &chan_config);
    printf("[INIT] ADC en canal %d (GPIO 34)\n",
POT_ADC_CHANNEL);
}

// Callback de envío ESP-NOW
static void espnow_envio_cb(const wifi_tx_info_t
*tx_info, esp_now_send_status_t status)
{
    g_envio_exitoso = (status ==
ESP_NOW_SEND_SUCCESS);
}

static void init_espnow(void)
{
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret
== ESP_ERR_NVS_NEW_VERSION_FOUND) {
        nvs_flash_erase();
        nvs_flash_init();
    }

    esp_netif_init();
    esp_event_loop_create_default();
```



```
wifi_init_config_t cfg =
WIFI_INIT_CONFIG_DEFAULT();
esp_wifi_init(&cfg);
esp_wifi_set_mode(WIFI_MODE_STA);
esp_wifi_start();

uint8_t mac_propia[6];
esp_wifi_get_mac(WIFI_IF_STA, mac_propia);
printf("[INIT] MAC MAESTRO:
%02X:%02X:%02X:%02X:%02X:%02X\n",
        mac_propia[0], mac_propia[1],
mac_propia[2],
        mac_propia[3], mac_propia[4],
mac_propia[5]);

esp_now_init();
esp_now_register_send_cb(espnow_envio_cb);

esp_now_peer_info_t peer_info = {};
memcpy(peer_info.peer_addr, MAC_ESCLAVO, 6);
peer_info.channel = 0;
peer_info.encrypt = false;
esp_now_add_peer(&peer_info);

printf("[INIT] ESP-NOW listo. Esclavo:
%02X:%02X:%02X:%02X:%02X:%02X\n",
        MAC_ESCLAVO[0], MAC_ESCLAVO[1],
MAC_ESCLAVO[2],
        MAC_ESCLAVO[3], MAC_ESCLAVO[4],
MAC_ESCLAVO[5]);
}

// FUNCIONES AUXILIARES

static float mapear(float valor, float in_min,
float in_max, float out_min, float out_max)
{
    return (valor - in_min) * (out_max -
out_min) / (in_max - in_min) + out_min;
}

static float limitar(float valor, float minimo,
float maximo)
{
    if (valor < minimo) return minimo;
    if (valor > maximo) return maximo;
    return valor;
}

// TAREAS DE FREERTOS
```

```
// TAREA 1: Lee el IMU y el potenciómetro (datos
crudos)
void tarea_adquisicion(void *parametro)
{
    printf("[TASK] Adquisición iniciada\n");
    uint8_t buffer[6]; // 6 bytes del
acelerómetro

    for (;;) {
        // Leer 6 bytes del acelerómetro desde
registro 0x3B
        mpu_read_bytes(0x3B, buffer, 6);

        g_accel_x = (int16_t)((buffer[0] << 8) |
buffer[1]);
        g_accel_y = (int16_t)((buffer[2] << 8) |
buffer[3]);
        g_accel_z = (int16_t)((buffer[4] << 8) |
buffer[5]);

        // Leer potenciómetro
        adc_oneshot_read(adc_handle,
POT_ADC_CHANNEL, &g_pot_raw);

vTaskDelay(pdMS_TO_TICKS(PERODO_ADQUISICION));
    }
}

// TAREA 2: Procesa datos, luego calcula ángulos
y finalmente genera duties
void tarea_procesamiento(void *parametro)
{
    printf("[TASK] Procesamiento iniciado\n");

    for (;;) {
        float ax = (float)g_accel_x;
        float ay = (float)g_accel_y;
        float az = (float)g_accel_z;

        // Calcular Pitch y Roll
        // Pitch: inclinación del eje X respecto
al plano horizontal
        float pitch_nuevo = atan2(ax, sqrt(ay *
ay + az * az)) * (180.0 / M_PI);
        // Roll: inclinación del eje Y respecto
al plano horizontal
        float roll_nuevo = atan2(ay, sqrt(ax *
ax + az * az)) * (180.0 / M_PI);

        // Filtro complementario
```

```

    g_pitch_filtrado = ALFA_FILTRO *
g_pitch_filtrado + (1.0 - ALFA_FILTRO) *
pitch_nuevo;

    g_roll_filtrado = ALFA_FILTRO *
g_roll_filtrado + (1.0 - ALFA_FILTRO) *
roll_nuevo;

    // Guardar sin filtrar para monitoreo
    g_pitch = pitch_nuevo;
    g_roll = roll_nuevo;

    // SERVO VERTICAL (ROLL = arriba/abajo)
    // Medido: ROLL va de -85 (abajo) a +57
    (arriba)
    float roll_limitado =
limitar(g_roll_filtrado, ROLL_MIN, ROLL_MAX);

    // Invertir si necesario
    if (INVERTIR_VERTICAL) {
        roll_limitado = ROLL_MIN + ROLL_MAX
- roll_limitado;
    }

    // Mapear ROLL al duty del servo
vertical
    float duty_v = mapear(roll_limitado,
ROLL_MIN, ROLL_MAX,

(float)DUTY_MIN_HOMBRO_V,
(float)DUTY_MAX_HOMBRO_V);
    g_paquete.duty_hombro_v =
(uint32_t)limitar(duty_v,

(float)DUTY_MIN_HOMBRO_V,
(float)DUTY_MAX_HOMBRO_V);

    // SERVO HORIZONTAL (PITCH =
derecha/izquierda)
    // Medido: PITCH va de -8
    (derecha/Cristo) a -25 (cruzado/pecho)
    float pitch_limitado =
limitar(g_pitch_filtrado, PITCH_MIN, PITCH_MAX);

    if (INVERTIR_HORIZONTAL) {
        pitch_limitado = PITCH_MIN +
PITCH_MAX - pitch_limitado;
    }

    float duty_h = mapear(pitch_limitado,
PITCH_MIN, PITCH_MAX,

```

```

(float)DUTY_MIN_HOMBRO_H,
(float)DUTY_MAX_HOMBRO_H);
    g_paquete.duty_hombro_h =
(uint32_t)limitar(duty_h,

(float)DUTY_MIN_HOMBRO_H,
(float)DUTY_MAX_HOMBRO_H);

    // SERVO DEL COD0 (Potenci6metro =
flexi6n)
    // Medido: POT≈830 = extendido, POT≈2022
= flexionado 135°
    float pot_limitado =
limitar((float)g_pot_raw, (float)POT_EXTENDIDO,
(float)POT_FLEXIONADO);

    // Mapear de POT a 6ngulo (830→0°,
2022→135°)
    float codo_ang = mapear(pot_limitado,
(float)POT_EXTENDIDO, (float)POT_FLEXIONADO,
COD0_ANGULO_MIN,
COD0_ANGULO_MAX);

    if (INVERTIR_COD0) {
        codo_ang = COD0_ANGULO_MIN +
COD0_ANGULO_MAX - codo_ang;
    }

    // Mapear 6ngulo del codo a duty
    float duty_c = mapear(codo_ang,
COD0_ANGULO_MIN, COD0_ANGULO_MAX,

(float)DUTY_MIN_COD0, (float)DUTY_MAX_COD0);
    g_paquete.duty_codo =
(uint32_t)limitar(duty_c,

(float)DUTY_MIN_COD0, (float)DUTY_MAX_COD0);

vTaskDelay(pdMS_TO_TICKS(PERIOD0_PROCESAMIENTO))
;
    }
}

// TAREA 3: Envía el paquete al esclavo por ESP-
NOW
void tarea_envio(void *parametro)
{
    printf("[TASK] Envío ESP-NOW iniciado\n");

    for (;;) {

```

```

        esp_err_t res =
        esp_now_send(MAC_ESCLAVO,

                                (uint8_t

*) &g_paquete,

sizeof(paquete_servos_t));
        if (res != ESP_OK) {
            printf("[ERROR] Fallo al enviar\n");
        }

vTaskDelay(pdMS_TO_TICKS(PERIODO_ENVIO));
    }
}

// TAREA 4: Imprime por serial para monitoreo y
calibración
void tarea_monitor(void *parametro)
{
    printf("[TASK] Monitor iniciado\n");

    for (;;) {
        printf("PITCH:%6.1f (f:%6.1f) |
ROLL:%6.1f (f:%6.1f) | POT:%4d | "
            "Duty[Hh:%5lu Hv:%5lu Co:%5lu] |
TX:%s\n",

            g_pitch, g_pitch_filtrado,
            g_roll, g_roll_filtrado,
            g_pot_raw,
            (unsigned
long)g_paquete.duty_hombro_h,
            (unsigned
long)g_paquete.duty_hombro_v,
            (unsigned
long)g_paquete.duty_codo,
            g_envio_exitoso ? "OK" : "FAIL");

vTaskDelay(pdMS_TO_TICKS(PERIODO_MONITOR));
    }
}

//FUNCIÓN PRINCIPAL

void app_main(void)
{
    printf("\nBRAZO ROBOT - ESP32 MAESTRO
(CALIBRADO)\n\n");

    printf("Mapeo de sensores:\n");
    printf("    ROLL    [%.0f a %.0f] = Servo
VERTICAL (arriba/abajo)\n", ROLL_MIN,
ROLL_MAX);

```

```

        printf("    PITCH [%.0f a %.0f] = Servo
HORIZONTAL (derecha/izquierda)\n", PITCH_MIN,
PITCH_MAX);
        printf("    POT    [%d a %d] = Servo CODO
(extendido/flexionado)\n\n", POT_EXTENDIDO,
POT_FLEXIONADO);

        init_i2c();
        init_mpu();
        init_adc();
        init_espnw();

        printf("\nCreando tareas FreeRTOS...\n");

        // Prioridad 4: Adquisición (la más alta →
datos frescos)
        xTaskCreate(tarea_adquisicion,
"Adquisicion", 4096, NULL, 4, NULL);

        // Prioridad 3: Procesamiento (calcula
ángulos y duties)
        xTaskCreate(tarea_procesamiento,
"Procesamiento", 4096, NULL, 3, NULL);

        // Prioridad 2: Envío ESP-NOW
        xTaskCreate(tarea_envio, "Envio", 4096,
NULL, 2, NULL);

        // Prioridad 1: Monitor serial (solo
informativo)
        xTaskCreate(tarea_monitor, "Monitor", 4096,
NULL, 1, NULL);

        printf("Maestro iniciado. Enviando datos al
esclavo...\n\n");
    }
}

```

Código Esclavo

```

/*
BRAZO ROBOT - ESP32 ESCLAVO (SERVOS)

Descripción:
    Este ESP32 recibe los valores de duty vía
ESP-NOW desde el maestro
    y los aplica a los 3 servos (2x MG996R hombro
+ 1x MG90S codo).
    Si pierde la conexión, mueve los servos a
posición central segura.

```

```
Protocolo: ESP-NOW (recepción inalámbrica punto a punto)
- Recibe paquetes con 3 valores de duty ya calculados
- Incluye timeout de seguridad por pérdida de señal
*/
```

```
#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/ledc.h"
#include "esp_wifi.h"
#include "esp_now.h"
#include "esp_netif.h"
#include "esp_event.h"
#include "nvs_flash.h"
#include "esp_timer.h"
```

```
///// VARIABLES DE CALIBRACIÓN /////
```

```
// Pines de los Servos
#define SERVO_HOMBRO_H_PIN 25 // GPIO para servo hombro horizontal (MG996R)
#define SERVO_HOMBRO_V_PIN 26 // GPIO para servo hombro vertical (MG996R)
#define SERVO_CODO_PIN 27 // GPIO para servo codo (MG90S)
```

```
// Configuración PWM de los servos
#define SERVO_FRECUENCIA 50
// 50 Hz
#define SERVO_DUTY_RES LEDC_TIMER_16_BIT
// Resolución 16 bits (0-65535)
```

```
#define SERVO_DUTY_MIN 2300 // Duty mínimo (pulso 0.5 ms)
#define SERVO_DUTY_MAX 8192 // Duty máximo (pulso 2.5 ms)
```

```
// Timeout de seguridad
#define TIMEOUT_SIN_DATOS_MS 2000 // ms sin recibir antes de ir a posición segura
```

```
// Periodos de las tareas (milisegundos)
#define PERIODO_SALIDA 20 // Cada cuánto se actualizan los servos
#define PERIODO_MONITOR 500 // Cada cuánto se imprime por serial
```

```
// ESTRUCTURA DEL PAQUETE DE DATOS
```

```
typedef struct {
    uint32_t duty_hombro_h; // Duty del servo horizontal del hombro
    uint32_t duty_hombro_v; // Duty del servo vertical del hombro
    uint32_t duty_codo; // Duty del servo del codo
} paquete_servos_t;
```

```
// VARIABLES GLOBALES
```

```
// Paquete recibido del maestro (se actualiza en el callback de recepción)
paquete_servos_t g_paquete_recibido = {0, 0, 0};
```

```
// Marca de tiempo del último paquete recibido (milisegundos desde arranque)
```

```
volatile uint64_t g_ultimo_paquete_ms = 0;
```

```
// Contador de paquetes recibidos (para monitoreo)
```

```
volatile uint32_t g_paquetes_recibidos = 0;
```

```
// Flag de conexión activa con el maestro
```

```
volatile bool g_conexion_activa = false;
```

```
// FUNCIONES DE INICIALIZACIÓN
```

```
// Devuelve el tiempo en milisegundos desde el arranque del programa
```

```
static uint64_t get_time_ms(void)
```

```
{
    return esp_timer_get_time() / 1000; // Microsegundos a milisegundos
}
```

```
// Callback: se ejecuta cada vez que llega un paquete por ESP-NOW
```

```
static void espnow_recepcion_cb(const esp_now_recv_info_t *info,
```

```
const uint8_t
```

```
*data,
```

```
int data_len)
```

```
{
```

```
    // Verificar que el paquete sea del tamaño correcto
```

```
    if (data_len == sizeof(paquete_servos_t)) {
```

```

        // Copiar los datos recibidos a la
variable global
        memcpy(&g_paquete_recibido, data,
sizeof(paquete_servos_t));
        g_ultimo_paquete_ms = get_time_ms(); //
Marcar tiempo de recepción
        g_paquetes_recibidos++;
Incrementar contador
        g_conexion_activa = true;
    }
}

// Inicializa WiFi + ESP-NOW para recepción
inalámbrica
static void init_espnw(void)
{
    // Inicializar NVS (requerido por WiFi)
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret
== ESP_ERR_NVS_NEW_VERSION_FOUND) {
        nvs_flash_erase();
        nvs_flash_init();
    }

    // Inicializar pila de red y loop de eventos
    esp_netif_init();
    esp_event_loop_create_default();

    // Configurar WiFi en modo estación (sin
conectar a router)
    wifi_init_config_t cfg =
WIFI_INIT_CONFIG_DEFAULT();
    esp_wifi_init(&cfg);
    esp_wifi_set_mode(WIFI_MODE_STA);
    esp_wifi_start();

    uint8_t mac_propia[6];
    esp_wifi_get_mac(WIFI_IF_STA, mac_propia);
    printf("\n");
    printf("  MAC DEL ESCLAVO\n");
    printf("  %02X:%02X:%02X:%02X:%02X:%02X\n",
        mac_propia[0], mac_propia[1],
mac_propia[2],
        mac_propia[3], mac_propia[4],
mac_propia[5]);
    printf("\n");
    printf("  {0x%02X, 0x%02X, 0x%02X, 0x%02X,
0x%02X, 0x%02X}\n",
        mac_propia[0], mac_propia[1],
mac_propia[2],
        mac_propia[3], mac_propia[4],
mac_propia[5]);

```

```

        // Inicializar ESP-NOW y registrar callback
de recepción
        esp_now_init();

        esp_now_register_recv_cb(espnw_recepcion_cb);

        printf("ESP-NOW inicializado. Esperando
datos del maestro...\n");
    }

    // Configura un canal PWM para un servo
específico
    static void configurar_servo_pwm(int gpio,
ledc_channel_t canal, ledc_timer_t timer)
    {
        // Configurar el timer del LEDC para el
servo (frecuencia y resolución)
        ledc_timer_config_t timer_conf = {
            .speed_mode = LEDC_LOW_SPEED_MODE,
            .timer_num = timer,
            .duty_resolution = SERVO_DUTY_RES, //
16 bits
            .freq_hz = SERVO_FRECUENCIA, //
50 Hz
            .clk_cfg = LEDC_AUTO_CLK,
        };
        ledc_timer_config(&timer_conf);

        // Configurar el canal PWM vinculado al GPIO
del servo
        ledc_channel_config_t ch_conf = {
            .speed_mode = LEDC_LOW_SPEED_MODE,
            .channel = canal,
            .timer_sel = timer,
            .intr_type = LEDC_INTR_DISABLE,
            .gpio_num = gpio,
            .duty = 0, // Iniciar apagado
            .hpoint = 0,
        };
        ledc_channel_config(&ch_conf);
    }

    // Inicializa los 3 canales PWM para los 3
servos
    static void init_servos(void)
    {
        // Los 3 servos usan el mismo Timer 0 pero
canales distintos
        configurar_servo_pwm(SERVO_HOMBRO_H_PIN,
LEDC_CHANNEL_0, LEDC_TIMER_0);

```



```

    configurar_servo_pwm(SERVO_HOMBRO_V_PIN,
LEDC_CHANNEL_1, LEDC_TIMER_0);
    configurar_servo_pwm(SERVO_CODO_PIN,
LEDC_CHANNEL_2, LEDC_TIMER_0);

    printf("[INIT] Servos PWM: Hh=GPIO%d,
Hv=GPIO%d, Codo=GPIO%d\n",
        SERVO_HOMBRO_H_PIN,
SERVO_HOMBRO_V_PIN, SERVO_CODO_PIN);
}

// FUNCIONES AUXILIARES

// Limita un valor de duty al rango válido del
servo
static uint32_t limitar_duty(uint32_t duty)
{
    if (duty < SERVO_DUTY_MIN) return
SERVO_DUTY_MIN;
    if (duty > SERVO_DUTY_MAX) return
SERVO_DUTY_MAX;
    return duty;
}

// Calcula el duty de la posición central (punto
medio entre min y max)
static uint32_t duty_central(void)
{
    return (SERVO_DUTY_MIN + SERVO_DUTY_MAX) /
2;
}

// Actualiza el duty de un servo
static void escribir_servo(ledc_channel_t canal,
uint32_t duty)
{
    ledc_set_duty(LEDC_LOW_SPEED_MODE, canal,
duty); // Establecer duty
    ledc_update_duty(LEDC_LOW_SPEED_MODE,
canal); // Aplicar cambio
}

// TAREAS DE FREERTOS

// TAREA 1: Aplica los duties recibidos a los
servos
void tarea_salida(void *parametro)
{
    printf("[TASK] Tarea de salida iniciada\n");

    for (;;) {
        uint64_t ahora = get_time_ms();

```

```

        uint64_t tiempo_sin_datos = ahora -
g_ultimo_paquete_ms;

        // Verificar timeout (¿se perdió la
conexión con el maestro?)
        if (g_ultimo_paquete_ms == 0 ||
tiempo_sin_datos > TIMEOUT_SIN_DATOS_MS) {
            // SIN CONEXIÓN: mover todos los
servos a posición central (segura)
            g_conexion_activa = false;
            uint32_t centro = duty_central();
            escribir_servo(LEDC_CHANNEL_0,
centro); // Hombro horizontal al centro
            escribir_servo(LEDC_CHANNEL_1,
centro); // Hombro vertical al centro
            escribir_servo(LEDC_CHANNEL_2,
centro); // Codo al centro
        } else {
            // CONEXIÓN ACTIVA: aplicar los
valores recibidos del maestro
            // Se limitan por seguridad antes de
aplicar
            escribir_servo(LEDC_CHANNEL_0,
limitar_duty(g_paquete_recibido.duty_hombro_h));
            escribir_servo(LEDC_CHANNEL_1,
limitar_duty(g_paquete_recibido.duty_hombro_v));
            escribir_servo(LEDC_CHANNEL_2,
limitar_duty(g_paquete_recibido.duty_codo));
        }

vTaskDelay(pdMS_TO_TICKS(PERODO_SALIDA));
    }

// TAREA 2: Imprime por serial para monitoreo y
depuración
void tarea_monitor(void *parametro)
{
    printf("[TASK] Tarea de monitor
iniciada\n");
    uint32_t paquetes_anterior = 0; // Para
calcular paquetes por segundo

    for (;;) {
        // Calcular tasa de paquetes recibidos
por segundo
        uint32_t paq_actual =
g_paquetes_recibidos;
        uint32_t paq_por_seg = (paq_actual -
paquetes_anterior) * (1000 / PERODO_MONITOR);
        paquetes_anterior = paq_actual;

```

```

        if (g_conexion_activa) {
            printf("RX OK | Duty[Hh:%5lu Hv:%5lu
Co:%5lu] | %lu paq/s | Total:%lu\n",
                (unsigned
long)g_paquete_recibido.duty_hombro_h,
                (unsigned
long)g_paquete_recibido.duty_hombro_v,
                (unsigned
long)g_paquete_recibido.duty_codo,
                (unsigned long)paq_por_seg,
                (unsigned long)paq_actual);
        } else {
            printf("SIN CONEXION | Servos en
posicion central | Esperando maestro...\n");
        }

vTaskDelay(pdMS_TO_TICKS(PERIODO_MONITOR));
    }
}

// FUNCIÓN PRINCIPAL

void app_main(void)
{
    printf("\nBRAZO ROBOT - ESP32 ESCLAVO
(SERVOS)\n");

    // Inicializar periféricos
    init_espnw(); // WiFi + ESP-NOW (primero
para mostrar la MAC)
    init_servos(); // PWM para los 3 servos

    // Mover servos a posición central al
arrancar (posición segura)
    uint32_t centro = duty_central();
    escribir_servo(LED_CHANNEL_0, centro);
    escribir_servo(LED_CHANNEL_1, centro);
    escribir_servo(LED_CHANNEL_2, centro);
    printf("[INIT] Servos en posicion central
inicial\n");

    printf("\nCreando tareas FreeRTOS...\n");

    // Prioridad 3: Salida (la más alta porque
mover servos es lo crítico)
    xTaskCreate(tarea_salida, "Salida", 2048,
NULL, 3, NULL);

```

```

// Prioridad 1: Monitor serial (solo
informativo)
    xTaskCreate(tarea_monitor, "Monitor", 4096,
NULL, 1, NULL);

    printf("Esclavo iniciado. Esperando paquetes
del maestro...\n\n");
}

```

Introducción y Objetivos (10%)	10/10
Marco Teórico (10%)	10/10
Materiales y Diagramas (10%)	8/10
Código Implementado (30%)	10/10
Resultados y Análisis (30%)	9/10
Conclusiones y Presentación (10%)	10/10
Calificación: 95	